

## Feature Separation Analysis

```
feature_separation_current <- sdf_sql(
  sc,
  "
SELECT
  label_3m,
  COUNT(*) AS parties,

  /* -----
  Deterioration
  ----- */
  AVG(deterioration_acceleration)
  AS deterioration_mean,

  percentile_approx(
    deterioration_acceleration, 0.25
  ) AS deterioration_p25,

  percentile_approx(
    deterioration_acceleration, 0.50
  ) AS deterioration_median,

  percentile_approx(
    deterioration_acceleration, 0.75
  ) AS deterioration_p75,

  /* -----
  Adverse behavior
  ----- */
  AVG(adverse_acceleration)
  AS adverse_mean,

  percentile_approx(
    adverse_acceleration, 0.50
  ) AS adverse_median,

  /* -----
  Max DPD jump
  ----- */
  AVG(max_dpd_jump_recent_3m)
  AS max_dpd_mean,

  percentile_approx(
    max_dpd_jump_recent_3m, 0.50
  ) AS max_dpd_median,
```

```

percentile_approx(
    max_dpd_jump_recent_3m, 0.95
) AS max_dpd_p95,

/* -----
   Active accounts
   ----- */
AVG(active_accounts_change_vs_baseline)
    AS active_change_mean,

percentile_approx(
    active_accounts_change_vs_baseline, 0.25
) AS active_change_p25,

percentile_approx(
    active_accounts_change_vs_baseline, 0.50
) AS active_change_median,

percentile_approx(
    active_accounts_change_vs_baseline, 0.75
) AS active_change_p75,

AVG(active_accounts_changed_flag * 1.0)
    AS active_changed_pct,

/* -----
   Balance
   ----- */
AVG(balance_change_model)
    AS balance_change_mean,

percentile_approx(
    balance_change_model, 0.50
) AS balance_change_median,

AVG(log1p_abs_balance_change_model)
    AS logabs_balance_mean,

percentile_approx(
    log1p_abs_balance_change_model, 0.50
) AS logabs_balance_median,

AVG(balance_change_available_flag * 1.0)
    AS balance_available_pct,

```

```

/* -----
Past due
----- */
AVG(past_due_change_model)
  AS pastdue_change_mean,

percentile_approx(
  past_due_change_model, 0.50
) AS pastdue_change_median,

AVG(past_due_change_available_flag * 1.0)
  AS pastdue_available_pct,


/* -----
Delinquency
----- */
AVG(delinquency_share_change_model)
  AS delinquency_change_mean,

percentile_approx(
  delinquency_share_change_model, 0.50
) AS delinquency_change_median,

AVG(delinquency_change_available_flag * 1.0)
  AS delinquency_available_pct,


/* -----
Utilization
----- */
AVG(avg_utilization_change_model)
  AS avg_util_change_mean,

percentile_approx(
  avg_utilization_change_model, 0.50
) AS avg_util_change_median,

AVG(weighted_utilization_change_model)
  AS weighted_util_change_mean,

percentile_approx(
  weighted_utilization_change_model, 0.50
) AS weighted_util_change_median,

AVG(utilization_change_available_flag * 1.0)
  AS utilization_available_pct

```

```
FROM bu_frd.model_features_ab_final_2022_2025
```

```
GROUP BY label_3m
```

```
ORDER BY label_3m
```

```
"
```

```
) %>%
```

```
collect()
```

```
feature_separation_current
```

## 2- Separación de las features Z

```
feature_separation_z <- sdf_sql(
```

```
sc,
```

```
"
```

```
SELECT
```

```
label_3m,
```

```
COUNT(*) AS parties,
```

```
/* Active account Z */
```

```
AVG(active_accounts_z_model)
```

```
AS active_z_mean,
```

```
percentile_approx(
```

```
active_accounts_z_model, 0.25
```

```
) AS active_z_p25,
```

```
percentile_approx(
```

```
active_accounts_z_model, 0.50
```

```
) AS active_z_median,
```

```
percentile_approx(
```

```
active_accounts_z_model, 0.75
```

```
) AS active_z_p75,
```

```
AVG(active_accounts_z_available_flag * 1.0)
```

```
AS active_z_available_pct,
```

```
/* Average utilization Z */
```

```
AVG(avg_utilization_z_model)
```

```
AS avg_util_z_mean,
```

```
percentile_approx(
```

```
avg_utilization_z_model, 0.25
```

```

) AS avg_util_z_p25,

percentile_approx(
    avg_utilization_z_model, 0.50
) AS avg_util_z_median,

percentile_approx(
    avg_utilization_z_model, 0.75
) AS avg_util_z_p75,

AVG(avg_utilization_z_available_flag * 1.0)
    AS avg_util_z_available_pct,

/* Weighted utilization Z */
AVG(weighted_utilization_z_model)
    AS weighted_util_z_mean,

percentile_approx(
    weighted_utilization_z_model, 0.25
) AS weighted_util_z_p25,

percentile_approx(
    weighted_utilization_z_model, 0.50
) AS weighted_util_z_median,

percentile_approx(
    weighted_utilization_z_model, 0.75
) AS weighted_util_z_p75,

AVG(weighted_utilization_z_available_flag * 1.0)
    AS weighted_util_z_available_pct

FROM bu_frd.model_features_ab_final_2022_2025

GROUP BY label_3m

ORDER BY label_3m
"
) %>%
collect()

feature_separation_z

```

#### 4. Availability vs Value Signal

```
feature_availability_signal <- sdf_sql(  
  sc,  
  "  
SELECT  
  feature,  
  availability_status,  
  
  COUNT(*) AS parties,  
  
  SUM(label_3m) AS positive_cases,  
  
  AVG(label_3m * 1.0)  
  AS positive_rate  
  
FROM (  
  
  /* Balance */  
  SELECT  
    label_3m,  
    'balance_change' AS feature,  
    CASE  
      WHEN balance_change_available_flag = 1  
      THEN 'AVAILABLE'  
      ELSE 'NOT_AVAILABLE'  
    END AS availability_status  
  
  FROM bu_frd.model_features_ab_final_2022_2025  
  
  UNION ALL  
  
  /* Past due */  
  SELECT  
    label_3m,  
    'past_due_change',  
    CASE  
      WHEN past_due_change_available_flag = 1  
      THEN 'AVAILABLE'  
      ELSE 'NOT_AVAILABLE'  
    END  
  
  FROM bu_frd.model_features_ab_final_2022_2025  
  
  UNION ALL  
  
  /* Delinquency */  
  SELECT
```

```
label_3m,  
'delinquency_change',  
CASE  
    WHEN delinquency_change_available_flag = 1  
    THEN 'AVAILABLE'  
    ELSE 'NOT_AVAILABLE'  
END
```

FROM bu\_frd.model\_features\_ab\_final\_2022\_2025

UNION ALL

/\* Utilization \*/

```
SELECT  
    label_3m,  
    'utilization_change',  
    CASE  
        WHEN utilization_change_available_flag = 1  
        THEN 'AVAILABLE'  
        ELSE 'NOT_AVAILABLE'  
    END
```

FROM bu\_frd.model\_features\_ab\_final\_2022\_2025

UNION ALL

/\* Active accounts Z \*/

```
SELECT  
    label_3m,  
    'active_accounts_z',  
    CASE  
        WHEN active_accounts_z_available_flag = 1  
        THEN 'AVAILABLE'  
        ELSE 'NOT_AVAILABLE'  
    END
```

FROM bu\_frd.model\_features\_ab\_final\_2022\_2025

UNION ALL

/\* Average utilization Z \*/

```
SELECT  
    label_3m,  
    'avg_utilization_z',  
    CASE  
        WHEN avg_utilization_z_available_flag = 1  
        THEN 'AVAILABLE'  
        ELSE 'NOT_AVAILABLE'
```

END

FROM bu\_frd.model\_features\_ab\_final\_2022\_2025

UNION ALL

/\* Weighted utilization Z \*/

SELECT

label\_3m,

'weighted\_utilization\_z',

CASE

WHEN weighted\_utilization\_z\_available\_flag = 1

THEN 'AVAILABLE'

ELSE 'NOT\_AVAILABLE'

END

FROM bu\_frd.model\_features\_ab\_final\_2022\_2025

) x

GROUP BY

feature,

availability\_status

ORDER BY

feature,

availability\_status

"

) %>%

collect()

feature\_availability\_signal